

A Distributed Architecture for Training and Inference of RandomForest Models

Elisa Marzioli
 Department of Information Technology
 University of Rome Tor Vergata
 Rome, Italy
 elisa.marzioli@students.uniroma2.eu

Abstract—This paper presents a fault tolerant distributed architecture for training and inference of RandomForest models, evaluated on two different tasks of classification and regression. A model-parallel approach is implemented through a master-worker system that parallelizes training at the level of individual trees: each worker trains a subset of a single logical forest's trees, each tree drawing its own bootstrap sample from the full, shared training set. Inference redistributes tree indices across live workers and aggregates on the master via soft voting (classification) or averaging (regression), following the same scheme a single-machine RandomForest uses internally. Finally, a performance evaluation and comparison with the non-distributed baseline model is carried on. Following a deployment on AWS to ensure fault-tolerance and computational power, the system reaches 99.05% accuracy on classification (SDSS DR14, distributed) against 99.10% for a non-distributed baseline. Results thus confirm that the distributed architecture implemented is to be considered statistically equivalent to a non-distributed baseline, while yielding clear indications of performance gain for large datasets, as a synthetic-data scalability study (10k–50k rows, 1–8 workers) shows substantial speedup: $5.4\times$ – $7.2\times$ at 8 workers depending on dataset size.

Keywords— *distributed machine learning, RandomForest, model parallelism, stateless service, gRPC, AWS, fault tolerance, performance analysis*

I. INTRODUCTION

As datasets become increasingly greater in size and complexity, the ability to train Machine Learning models on their data is slowed down by the constraints of a single machine. For this reason, studies that explore distributed training present it as a solution that may provide useful benefits in terms of reduced training time at the cost of minimal losses of accuracy, that have nonetheless been tackled in multiple relevant studies in the past decades [1].

RandomForest appears as the perfect fit for distributed learning: as an ensemble of independent, uncorrelated trees trained via a bagging technique, which by its nature requires its workload to be split across nodes with little coordination, and partial results to be aggregated after the fact without need to synchronize during training. The bagging technique requires in fact a first “bootstrap” phase, where for each tree a so called *bootstrapped dataset* (a dataset randomly sampled from the original dataset, meant to represent a unique sample of the true distribution) is used to train the tree, followed by an aggregation phase where results from the uncorrelated trees are aggregated into a final prediction, resulting in a variance reduction for the final model.

Because every tree, both distributed and not, draws its bootstrap sample from the same full training set, the distributed model is expected to be a statistically equivalent estimator to the non-distributed baseline given the same total

tree count. Compared to the baseline, we expect reduced training and inference time as data or tree count grows, at the cost of coordination overhead (network round-trips, per-tree scheduling, result aggregation) which may be more visible in datasets of smaller size.

II. ARCHITECTURE

A. The system

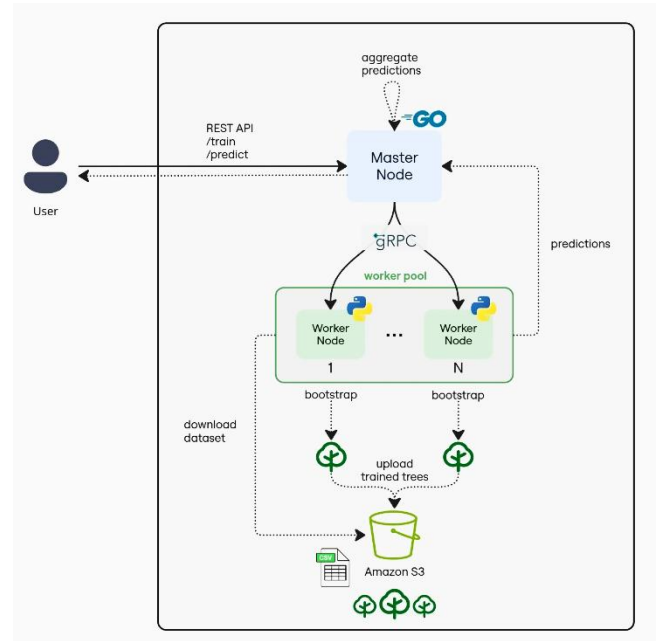


Fig. 1. Model architecture of the distributed RandomForest system

A user contacts a master node to query RandomForest models, requesting training of a certain-sized model using specific parameters, or inference on previously trained models providing a set of features. The master orchestrates a pool of workers, responsible for the distributed training and inference using subsets of trees of the forest. The master is also responsible for aggregating the partial results coming from the worker, in order to send an appropriate response to the client.

For training, the master persists request metadata to a shared storage, assigns trees to workers by distributing indices, and dispatches per-tree training in parallel. Each tree trains on a bootstrap sample of the full training set, extracted by the worker, and uploads to the shared storage as soon as it's done.

For inference, upon receiving a request to use a specific model, the master analyses the trained trees available for that model on the shared storage and splits the tree indices across the currently healthy workers, collects one prediction per tree,

and aggregates via soft voting (for classification) or averaging (for regression). Aggregation is performed only once (no partial aggregation on workers), in both cases, to reduce error.

B. Design Patterns

- Master-worker, centralized orchestration: the master decides partitioning, participation, and aggregation; workers don't carry any orchestration logic.
- Polyglot Microservices: master and workers are implemented in different programming languages, each chosen according to its functions; communication despite independence of the services is guaranteed thanks to gRPC.
- Scatter-gather: both flows scatter to every healthy worker in parallel and gather results for aggregation.
- Stateless workers: no state survives between gRPC calls; all durable state (metadata, trees) lives on shared storage. Because of this, a worker that trained a tree is not necessarily the one that later serves it, and a worker crash loses only in-flight work.
- Server-side discovery (client to master): use of a simple load balancer.
- Cold standby for the master: an autoscaler (AWS ASG) is responsible for keeping the master alive. Reconciliation logic was implemented such that, on boot, a new master whatever a previous master left unfinished.

III. IMPLEMENTATION

A. Infrastructure

The master exposes a REST API to clients and talks to workers over gRPC, using a shared Protocol Buffers contract. The gRPC protocol was chosen for its support to polyglot services, which allowed us to program the master in GO, as it is responsible for orchestration of parallel tasks easily implemented with goroutines, and the workers in Python, for the extensive support of ML libraries needed for the machine learning tasks. Workers use scikit-learn for tree training/inference. Both read and write a shared S3-compatible store: Amazon S3 on AWS, MinIO locally via Docker Compose.

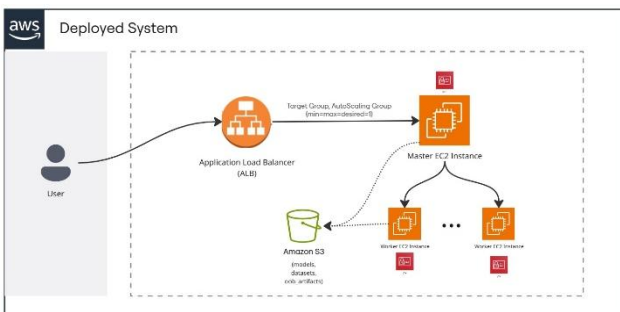


Fig. 2. AWS Deployment: components and interactions

The AWS deployment adds EC2 to deploy instances, an Application Load Balancer and Target Group in front of the master, and an Auto Scaling Group holding exactly one master instance alive at all times to guarantee fault tolerance.

B. API and contracts

The REST API (Gin, internal/api) exposes the following methods:

- POST /train: (dataset_url, task_type, target_column, n_estimators, optional hyperparameters). Asynchronous call, returns 202 with a fresh model_id and status="training" immediately; training runs asynchronously from the request context, so neither client disconnect nor a later master crash affects an already-started job (recovery is reconciliation, not context propagation).
- GET /models/{model_id} Reports model status (not_found / training / ready / failed) by inspecting S3.
- POST /predict/{model_id} (synchronous); returns once every live worker has answered (or failed) and results are aggregated.
- GET /health: Used by the ALB Target Group.

Regarding gRPC (api/proto/worker/v1/worker.proto), service Worker exposes: Train, Predict, Health. TrainRequest carries the full dataset URL, task type, target column, a hyperparameters map, and an explicit list of tree indices assigned to that worker. PredictRequest carries the same kind of explicit index list, letting the master reassign a worker's trees to a replacement mid-request.

Hyperparameters are sent from the master to the worker through a generic map. The master (internal/api/hyperparameters.go) validates every key against a configured allow-list (max_depth, max_features, min_samples_split, min_samples_leaf, criterion, max_leaf_nodes, class_weight) and checks each value's shape.

C. Fault tolerance

During training, each worker uploads a tree to S3 as soon as it is trained, rather than waiting for its full batch. If a worker crashes, only the tree in progress is lost; the master detects the missing index and reassigns it to another worker, up to a limited number of retries. If the master itself crashes, a replacement instance checks S3 on startup and resumes training only the missing trees under the original model ID. This is possible because each tree's training data is derived deterministically from the model ID and tree index, so a resumed tree is identical to the one that would have been produced without the interruption.

During inference, workers hold no state, so any worker can compute the prediction for any tree. If a worker fails while serving a request, its trees are reassigned within the same request. If some trees remain unreachable after retries, the result is still returned based on the trees that responded, with the missing indices reported; only a complete failure of all trees returns an error.

The master's availability is handled at the infrastructure level: it runs as a single EC2 instance behind a Load Balancer, with an Auto Scaling Group configured to keep exactly one instance running at all times. A failed instance is replaced automatically and registered once it passes its health check. This mechanism is only effective in combination with the S3-based recovery logic described above, without which a new master instance would have no record of in-progress work.

D. Machine Learning Model

1) Tasks

The study evaluates the system on two tasks: classification on SDSS DR14 (Sloan Digital Sky Survey, ~10,000 rows, 3-class STAR/GALAXY/QSO) [4] and regression on California housing (median_house_value from location, room counts, and income) [5]. A third, synthetic classification dataset (make_classification, column layout mirroring SDSS) is used for the scalability study (§VI.B), where dataset size must be controlled precisely.

Predictions are aggregated by soft voting: each tree returns its class-probability vector over the classes present in its own bootstrap sample, the coordinator sums probabilities per label and returns the arg-max, with ties resolved in sorted label order. This is the rule RandomForestClassifier.predict applies internally, which is what makes the distributed forest the same estimator as the single-machine baseline rather than an approximation of it. For regression the tree outputs are averaged.

A simple generic preprocessing is performed by the worker on each dataset, expecting the data to be already mostly preprocessed by the user.

2) Parameters

Following Breiman[1], max_features is $\sqrt{p}$ for classification and $p/3$ for regression; the split criterion is entropy (classification) and mean squared error (regression); trees are grown to full depth (max_depth=None), as the forest reduces variance by averaging rather than by pruning. Users may override seven per-tree hyperparameters (max_depth, max_features, min_samples_split, min_samples_leaf, criterion, max_leaf_nodes, class_weight); ensemble-only parameters (bootstrap, max_samples, oob_score) are not exposed, since sampling happens outside scikit-learn.

3) Metrics and evaluation

To obtain a test set, a standalone script (scripts/split.py) shuffles and splits a source dataset once (fixed seed, optional stratification, using an 80/20 proportion for train and test), uploading <name>_train.csv/<name>_test.csv on the shared storage. This keeps row order canonical for bootstrap sampling and guarantees the baseline sees identical data.

Accuracy (scripts/evaluation.py) is measured by a user run script by calling the real POST /predict once per held-out row compared against the baseline's in-memory scikit-learn predictions on the same rows. Classification reports accuracy and a confusion matrix; regression reports MSE/RMSE.

To enable validation of the RandomForest model, it's possible to compute the OOB error of each forest and get an estimate of its generalization error without using a validation set (scripts/evaluate_oob.py).

Bootstrap sampling uses a deterministic seed, sha256(model_id:tree_index) truncated to 32 bits (in scripts/bootstrap.py), for both the row draw and the tree's random_state. Because each tree's bootstrap seed is deterministic, a worker recomputes which rows were left out of its sample right after training, tests them, and uploads to the shared storage a small file containing predictions made by the tree on data points that don't belong to its bootstrapped dataset. A separate aggregation step on the master combines every tree's contribution, mirroring RandomForest(oob_score=True) without needing scikit-learn or the seed itself at aggregation time.

IV. TESTING

The system is tested with a Go suite that runs against fake gRPC workers and an in-memory S3, so no real cluster is needed. Unit tests cover the aggregation logic and small helper functions; integration tests cover training and inference under worker failures, and recovery after a master restart. HTTP-level tests cover the REST endpoints.

Master failure itself is not covered by a test suite; it is tested manually on AWS, by terminating the instance and checking that it is replaced and recovers correctly.

V. EXPERIMENTAL METHODS

All experiments compare the distributed system with a non-distributed baseline: a scikit-learn RandomForest trained on one machine with the same training set, preprocessing, hyperparameters and n_jobs=1, so that the two differ only in the distribution of the trees. Two environments were used. Locally, master, workers and a MinIO bucket run as Docker Compose containers on a single 16-core laptop, for testing purposes. On AWS, the master and each worker run on separate EC2 instances (t3.micro) with a real S3 bucket. Each dataset is split once into 80% training and 20% test (stratified for classification, fixed seed) by an offline script; the system only ever receives the training file.

Results will focus on the data obtained with AWS infrastructure, to emulate a real production environment.

All experiments were run with 8 worker nodes, except for the last one where the number of workers is variable.

A. Evaluation metrics

One forest of 100 trees is trained per model; the distributed one is scored by calling the real /predict endpoint once per test row, the baseline by scikit-learn directly. We report accuracy and the per-class confusion matrix for classification, RMSE for regression, and the out-of-bag estimate of the distributed forest.

B. Training and Inference time

The second experiment measures training and inference time on the same two tasks. Each model is trained and queried 100 times; the query is a fixed test row repeated identically, chosen so that the variance observed reflects the system rather than the input. Distributed training time is measured as a client

would see it, from POST /train to the first GET /models/{id} reporting ready; baseline training time starts from the same point, the dataset on S3.

C. Validation

The third experiment selects the forest size. On SDSS, forests of 10, 30, 50, 100, 150 and 200 trees are trained and their out-of-bag error and training time are recorded, to locate the point beyond which additional trees no longer improve accuracy.

D. Scalability analysis

The fourth experiment evaluates scalability with synthetic data. Three classification datasets of 10k, 20k and 50k rows are generated with `make_classification`, with eight features and three classes mirroring the structure of SDSS. With the forest size fixed at 100 trees, training and inference time are measured for 1, 2, 4 and 8 workers, ten runs per configuration, in both environments

VI. RESULTS

The following sections provide detailed numerical results for each of the experimental analyses described in the previous section, followed by interpretations of results.

A. Evaluation Metrics

Results of the comparison for the distributed model confirm that the two models can be considered equivalent.

TABLE I.

Model	Dataset	Accuracy
Distributed	Sloan Digital Sky Survey	0.9905
Baseline	Sloan Digital Sky Survey	0.991

Model	Dataset	RMSE ^a
Distributed	California Housing	52778.6
Baseline	California Housing	52663.5

^a. It's important to note here that the poor results on the Housing dataset are due to the simple preprocessing used (see *Limits*). Results are only meant to display equivalence of the models.

Fig.3 displays is the confusion matrix, which reports similar results and confirms the expected behaviour:

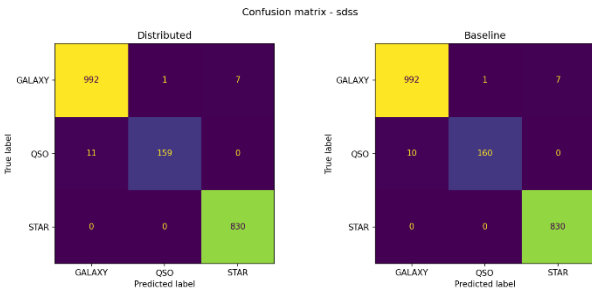


Fig. 3. Confusion matrix comparison (distributed vs baseline) – SDSS

Note that most of the misclassified instances belong to the smallest class (QSO), the least represented of the three in the dataset.

B. Training and Inference times

TABLE II.

Model	Training Time Comparison		
	Dataset	Mean (s)	Std. Dev.
Distributed	Sloan Digital Sky Survey	2.563166	0.295593
Baseline	Sloan Digital Sky Survey	1.851107	0.202267
Distributed	California Housing	3.154387	0.357860
Baseline	California Housing	4.308437	0.295710

TABLE III.

Model	Inference Time Comparison		
	Dataset	Mean (s)	Std. Dev.
Distributed	Sloan Digital Sky Survey	0.889753	0.303636
Baseline	Sloan Digital Sky Survey	0.009224	0.000761
Distributed	California Housing	1.012819	0.243330
Baseline	California Housing	0.012074	0.003647

Results confirm the initial expectation: it's mainly datasets of bigger sizes that benefit from the parallelization. This explains the fact that the distributed model only increases time efficiency for the housing dataset, which counts double the data as SDSS. Results also show that prediction times tend to increase seemingly alarmingly, which is, from our understanding, due to the expected communication overhead of the distributed model, rendered greater by the use of 8 workers.

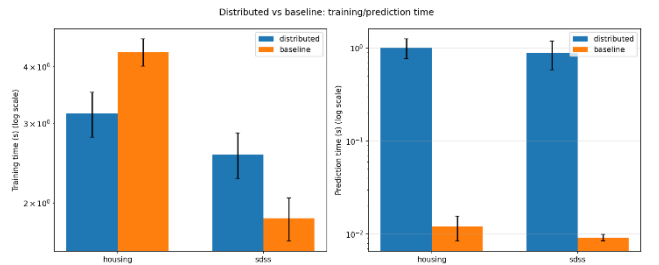


Fig. 4. Housing vs SDSS time comparison: shows greater gain for bigger dataset and impact of communication overhead in distributed inference

C. Validation

The results of the third experiment on the SDSS dataset show that the OOB Error rate tends to decrease as the number of estimators increases, converging to approximately 0.011 for the current configuration.

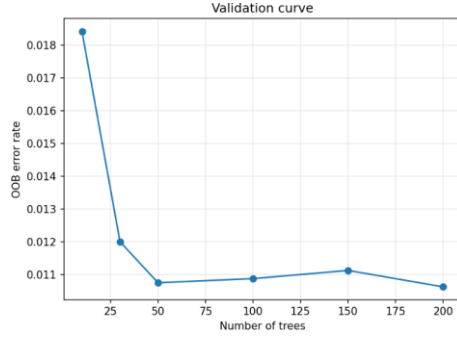


Fig. 5. Validation of the classifier: OOB vs number of estimators

D. Scalability

Table IV displays the results of the scalability analysis, including the mean time for both training and inference, and the relative confidence intervals (95%).

TABLE IV.

Dataset Size	W^c	Training Time		Inference Time	
		Mean(s)	CI	Mean(s)	CI
10k	1	18.535447	0.931135	5.411708	0.173042
10k	2	13.271997	3.287859	3.669757	0.929492
10k	4	7.698692	1.047118	2.228308	0.248832
10k	8	3.404906	0.398263	0.974414	0.033335
20k	1	25.953679	0.703407	5.324801	0.371601
20k	2	13.301792	0.666335	2.722298	0.146437
20k	4	7.489295	0.787937	1.633694	0.171306
20k	8	4.352065	0.500250	1.080680	0.196995
30k	1	54.786386	0.655698	6.790287	0.365456
30k	2	27.876207	0.491977	2.991416	0.203657
30k	4	14.259323	0.389205	1.655171	0.119731
30k	8	7.606877	0.290882	1.053862	0.180242

^b. Number of workers

The following results display the speedup for each configuration, computed as the ratio between the time taken by one worker (baseline) and the time taken by the selected configuration.

TABLE V.

Dataset Size	W^c	Training Time		Inference Time	
		Mean(s)	Speedup	Mean(s)	Speedup
10k	1	18.535447	1.00	5.411708	1.00
10k	2	13.271997	1.40	3.669757	1.47
10k	4	7.698692	2.41	2.228308	2.43
10k	8	3.404906	5.44	0.974414	5.55
20k	1	25.953679	1.00	5.324801	1.00
20k	2	13.301792	1.95	2.722298	1.96
20k	4	7.489295	3.47	1.633694	3.26

Dataset Size	W^c	Training Time		Inference Time	
		Mean(s)	Speedup	Mean(s)	Speedup
20k	8	4.352065	5.96	1.080680	4.93
30k	1	54.786386	1.00	6.790287	1.00
30k	2	27.876207	1.97	2.991416	2.27
30k	4	14.259323	3.84	1.655171	4.10
30k	8	7.606877	7.20	1.053862	6.44

^a. Number of workers

The same results are also displayed in Fig.6 and Fig.7, where it's immediately clear that, not only do all three configurations of dataset size benefit from the increase in the number of workers, but also that the performance gain increases as dataset size grows.

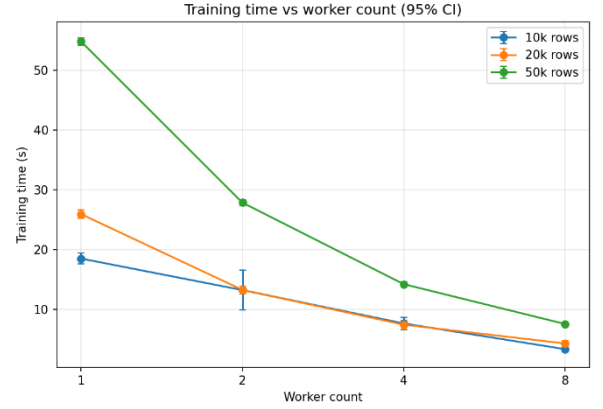


Fig. 6. Dataset size impact on training time vs number of workers (SDSS)

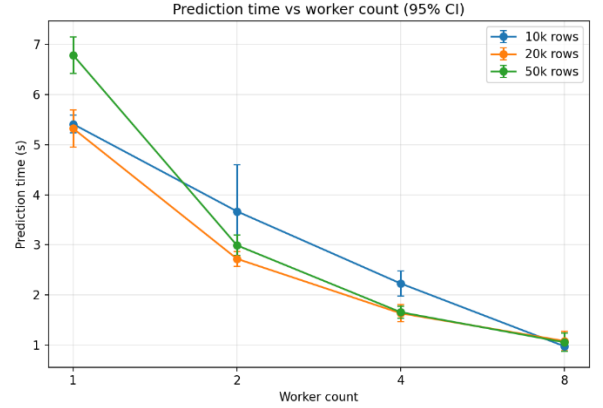


Fig. 7. Dataset size impact on inference time vs number of workers (SDSS)

VII. DISCUSSION

The results of the evaluation comparison support the base claim and validate the implementation of the architecture: the distributed forest is an equivalent estimator to the baseline, non-distributed one. The two models differ by one prediction out of 2000 on SDSS and by 0.2% RMSE on housing.

Our expectations that performance may deteriorate slightly due to the cost of the communication overhead was also confirmed by the results of the second experiment.

The results of the scalability analysis confirm that, as the size of the dataset increases, so does the benefit of the parallelization. It's relevant to highlight that, for the sake of the computational cost, the size of dataset was limited to 30k, a small number compared to production-grade scenarios. This implies that in real case scenarios the benefit of adopting a distributed architecture may surpass original expectations.

The analysis carried out by this study confirms that such a distributed architecture should only be preferred in case of datasets of greater dimensions.

VIII. THREATS TO VALIDITY

A threat to the validity of this study is that AWS Academy Learner Lab instances are shared, budget-constrained hardware, not dedicated — subject in principle to CPU credit throttling or neighbor contention.

IX. LIMITATIONS

It is important to keep in mind the current limitations of this work:

- OOB error and accuracy are only available via external scripts run after training, not as a live system feature.
- Categorical features are dropped rather than encoded, with a measurable cost on housing RMSE.
- Currently, accuracy is evaluated by calling `/predict` on each row of the dataset. This method is a clear limitation of the performance and clearly not scalable, despite allowing for a small scale study such as this.
- There is currently no garbage collection for old/abandoned model artifacts on S3.
- Master→worker topology is currently still a static, manually maintained address list, not a dynamic registry.

X. FUTURE WORK

If this work shall continue, such are the possible directions:

- Speed up accuracy evaluation at scale (using concurrent `/predict` calls, or sampling).
- Test on larger, more realistic datasets.
- Implement a Service Registry to allow for a more realistic dynamic discovery of worker nodes.
- Expose accuracy/OOB error as a live endpoint rather than an external script to be manually run by the user.

- Encode categorical features (e.g. one-hot), which requires extending the predict contract beyond a flat float array.
- Introduce garbage-collection of old/incomplete models on S3.
- A quantitative comparison against the data-parallelism architecture, both designs are fully implemented. Often described together in literature [2], but not as frequently actually compared.

XI. CONCLUSION

This paper presented a master-worker system distributing RandomForest training and inference at individual-tree granularity, and showed, both by construction and by measurement, that this reconstructs the same statistical estimator a single-machine RandomForest would produce: held-out accuracy, baseline accuracy, and OOB error agreed within 0.15 points on classification and 0.2% on regression. The scalability study showed an interesting speedup (5.4x–7.2x at 8 workers), which demonstrated to be increasing with dataset size, indicating that Big Datasets (> 100k) may benefit greatly from parallelization. The baseline comparison showed both outcomes are possible depending on workload: the baseline trained faster on the smaller SDSS forest, the distributed system faster on the larger housing one, which goes on to demonstrate how such distributed architectures provide a growing advantage for larger datasets.

In the times of Big Data, it is clear that distributed architectures will represent the new paradigm for training and inference of machine learning models on larger datasets. Research works exploring new architectures for distributed machine learning, ranging from model parallelism (here implemented), to data parallelism and federated learning, provide valuable insights on how to analyse and manage the increasing amounts of data flowing into our systems, comparing tradeoffs, techniques and solutions.

REFERENCES

- [1] Chan, P. K., & Stolfo, S. J. (1993, December). Experiments on multistrategy learning by meta-learning. In *Proceedings of the second international conference on information and knowledge management* (pp. 314-323).
- [2] Guilleme-Bert, M., & Teytaud, O. (2018). Exact distributed training: Random forest with billions of examples. *arXiv preprint arXiv:1804.06755*.
- [3] L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. <https://doi.org/10.1023/A:1010914300426>.
- [4] *Extracted dataset from Sloan Digital Sky Survey*. <https://www.kaggle.com/datasets/lucidlenn/sloan-digital-sky-survey>
- [5] *California Housing prices*. <https://www.kaggle.com/datasets/camnugent/california-housing-prices>